

UF9

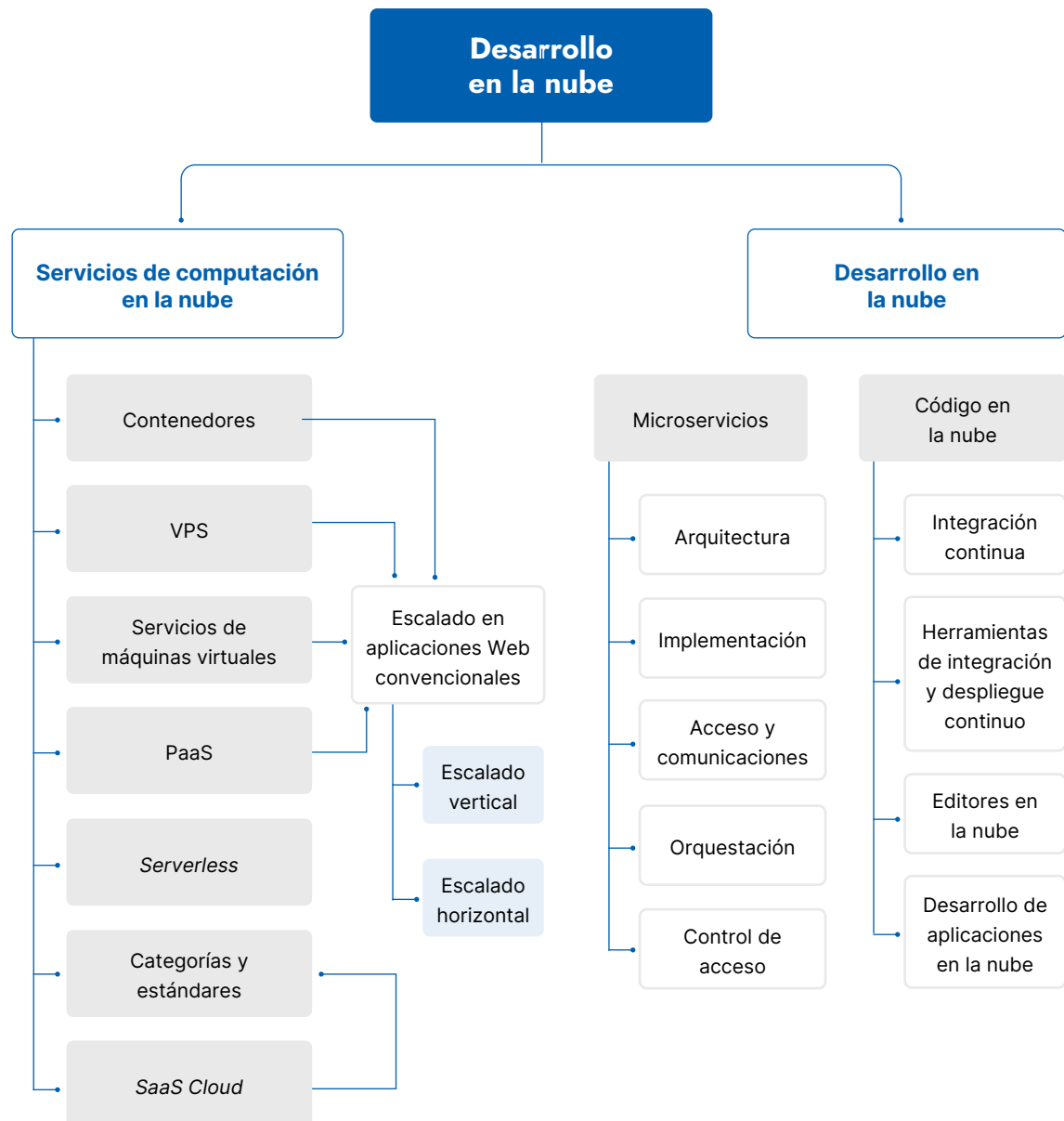
Servidores en la nube

Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	03
1. El diseño del modelo de negocio:	04
1.1. Servicios de máquinas virtuales y contenedores en AWS	04
1.2. Servicios de servidor privado virtual (VPS) y PaaS en AWS.....	05
1.3. Servicios de computación sin servidor (<i>serverless</i>).....	08
1.4. Servicios de computación en Azure.....	09
2. Escalado vertical y horizontal en la nube.....	10
2.1. Escalado vertical de portales web convencionales en la nube	10
2.2. Escalado horizontal de portales web convencionales en la nube ...	12
3. Servicios cloud para entornos de desarrollo	15
3.1. Introducción a los SaaS cloud.....	15
3.2. Principales SaaS de interés en los proveedores cloud	16
3.3. Servicio de almacenamiento masivo S3	18
3.4. Estándares de facto y decisión sobre proveedores	19
4. Microservicios	20
4.1. Arquitectura	20
4.2. Implementación.....	22
4.3. Acceso y comunicaciones	22
4.4. Orquestación.....	24
4.5. Control de acceso.....	25
5. Código en la nube	26
5.1. Integración continua	26
5.2. Herramientas de integración y despliegue continuos.....	27
5.3. Editores en la nube	29
5.4. Desarrollo de aplicaciones en la nube	29

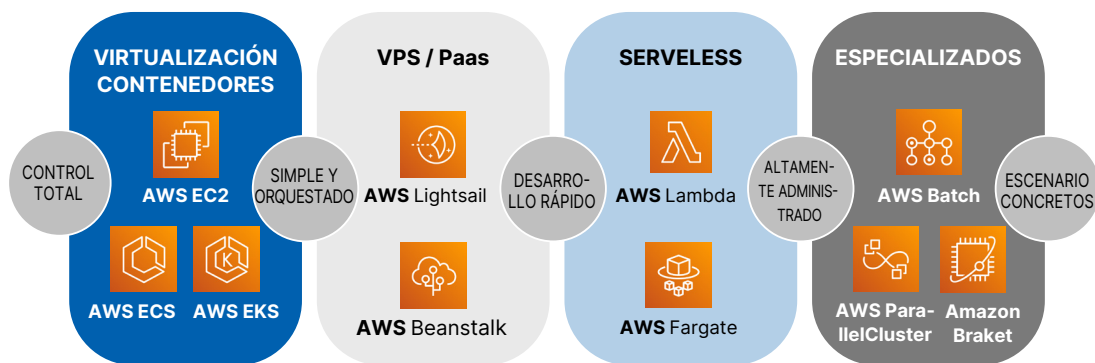
Mapa conceptual



01 El diseño del modelo de negocio:

En torno a 2006, en los comienzos del *cloud computing*, AWS ofrecía, como único actor, un servicio de virtualización de servidores a gran escala llamado Elastic Compute Cloud (EC2) y otro de almacenamiento masivo de objetos llamado Simple Storage Service (S3), con el objetivo de ofrecer a los clientes con CPD en propiedad un servicio similar, pero virtualizado y en pago por uso, con las ventajas de escalabilidad, ausencia de inversión inicial y menor coste.

Desde entonces, el extendido uso de aplicaciones web y de móviles, servicios multimedia masivos como Netflix o Spotify y el procesamiento de datos continuo relacionado con el *big data* han supuesto una gran evolución en los servicios de computación, más allá de ofrecer máquinas virtuales que sustituyan a las propias de un CPD.



Esquema 1. Panorama general clasificado de los servicios de computación de AWS.

Para este apartado, y en general para el resto de la unidad, vamos a tomar como referencia el proveedor *cloud* AWS para definir los conceptos y servicios, pero presentaremos también las equivalencias con Azure y sus matices. Como veremos más adelante al detallar los servicios, no es demasiado importante el proveedor concreto con el que se aprenda, dado que todos los conceptos y servicios tienen traducción en todos los proveedores *cloud*.

1.1. Servicios de máquinas virtuales y contenedores en AWS

En primer lugar, se dispone del **servicio de computación EC2 de virtualización de máquinas a gran escala**, que, como se ha comentado, es el servicio pionero de la computación en nube, y es realmente interesante introducirlo, dado que la mayor parte del resto de servicios está montado sobre instancias de EC2 de una u otra forma.

Para este módulo no se necesita profundizar en detalles; solo debemos quedarnos con el concepto de virtualización de máquinas nativa (HVM) que permite el máximo grado de control sobre las máquinas virtuales y los recursos que se lanzan (con pocas limitaciones o automatizaciones) y tiene el mayor grado de integración posible con la arquitectura y el resto de los servicios del proveedor.

En segundo lugar, existen los **servicios ECS y EKS de virtualización mediante contenedores**. Como se ha visto, es una forma de virtualización más ligera que la anterior, fundamentalmente indicada para la ejecución de aplicaciones en entornos aislados (algo clave, por ejemplo, en las modernas arquitecturas de microservicios).

El servicio ECS soporta el estándar de contenedores Docker de Windows y Linux, mientras que el servicio EKS se utiliza para el más complejo sistema de orquestación Kubernetes, creado originalmente por Google.

Desde 2018 se ha popularizado el término *container as a service*, o CaaS, como otra especialización del *cloud computing* (similar a DBaaS, IaaS, PaaS o SaaS, vistos en otras materias). En ella se ofrece un servicio de gestión integral de contenedores soportado de forma habitual por un clúster de máquinas subyacente, lo que evita tener que instalar y mantener la infraestructura, por un lado, y permite aprovechar la integración con el resto de los servicios y la arquitectura de la nube.

Sin entrar en muchos detalles, podemos indicar que en la práctica se suele utilizar ECS cuando se necesita un servicio de contenedores relativamente simple para migrar a la nube aplicaciones más o menos grandes contenerizadas en Docker, cuya gestión y orquestación seguirá los estándares de propios de AWS, «atándose» a su tecnología. No tiene coste en sí, sino que se paga por las máquinas subyacentes que soportan el clúster donde se despliega.

En contraposición, se suele utilizar EKS cuando se requiere un despliegue más complejo que utilice Kubernetes (normalmente, sobre contenedores Docker también), que es el estándar líder de la industria, con API *open-source* y una comunidad y un ecosistema vibrantes alrededor. Está claro que se asumirá una mayor complejidad en la implementación, pero a cambio se usa tecnología estándar no dependiente de AWS y se garantiza una mayor compatibilidad para migraciones a otros proveedores. Tiene un coste adicional por cada clúster que se da de alta, aparte de las máquinas subyacentes.

1.2. Servicios de servidor privado virtual (VPS) y PaaS en AWS

Una simplificación más accesible de los anteriores es el **servicio Lightsail**, que provee máquinas virtuales en formato de VPS (servidor privado virtual) y contenedores simples Docker solo para Linux a precios muy reducidos, utilizando grandes máquinas, compartidas

entre muchos clientes. En contrapartida, las características se reducen a planes muy cerrados y se pierde gran parte del control, características especiales y flexibilidad de los servicios anteriores.

En el ámbito de esta asignatura, puede resultar interesante para desplegar portales en pila LAMP (Linux-Apache-Php-MySQL/MariaDB) a un coste muy asequible y con *blueprints* muy automatizadas, que pueden compensar en muchas situaciones la pérdida de control citada.

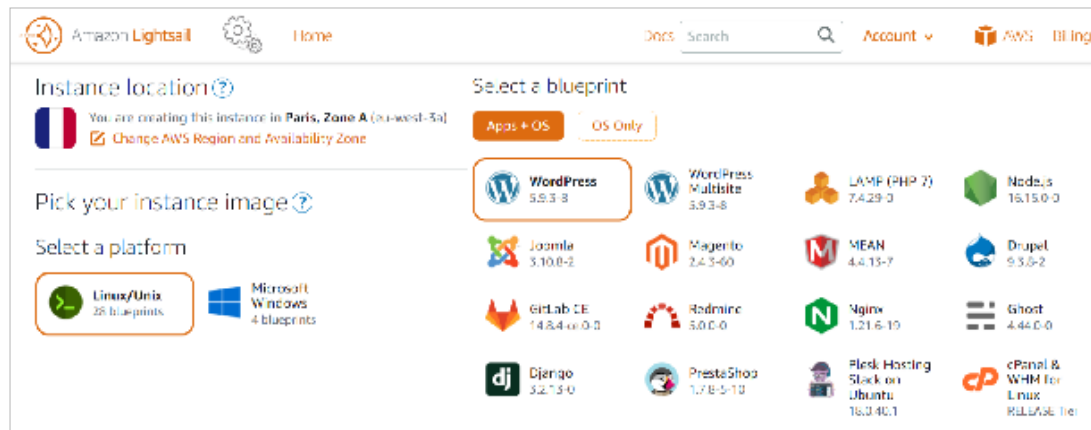


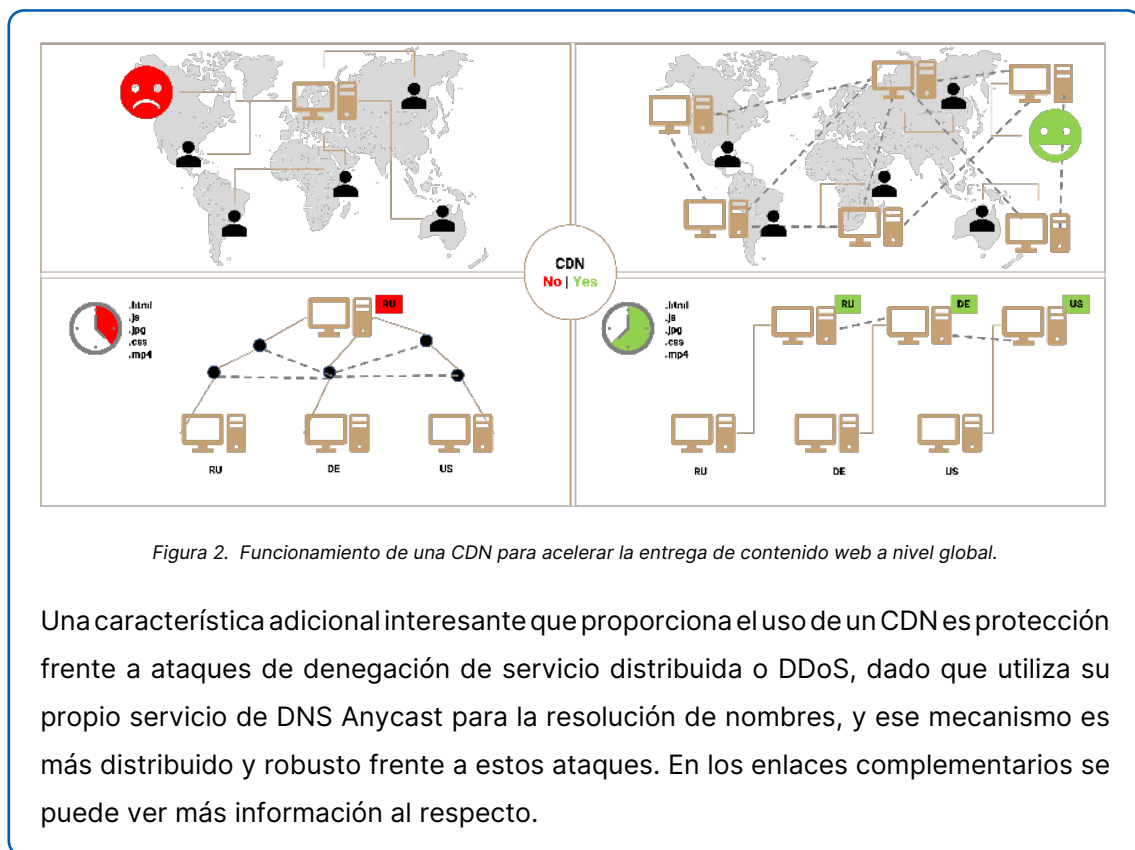
Figura 1. Lanzamiento de un VPS administrado en AWS Lightsail.

Es un servicio pensado en su momento para competir con las empresas de *hosting* convencionales, pero que en los últimos años ha aumentado su interés, al incorporar características avanzadas tales como balanceadores de carga, autoescalado o CDN (*content distribution networks*). Es muy recomendable para portales o aplicaciones simples y pequeñas empresas, y con las últimas características puede tener un ámbito mucho mayor, con un coste reducido.

Es un servicio muy interesante, pero, lamentablemente, hoy en día está fuera de alcance de las cuentas de AWS Academy, de forma que es necesario el uso de una cuenta de capa gratuita.

Siempre que hablamos de aplicaciones web y proveedores *cloud*, los servidores estarán ubicados necesariamente en una región concreta de su infraestructura, y ello puede causar en ocasiones problemas de latencia, pérdida de paquetes, lentitud en descargas y, en general, un empobrecimiento de la experiencia del usuario.

Para resolver este problema, se utiliza lo que se denomina «red de distribución de contenidos» o CDN, que es un servicio que permite ofrecer los contenidos estáticos de una aplicación desde ubicaciones que están más cercanas geográficamente al visitante que entra a la web. Este servicio es implementado por todos los proveedores *cloud* (AWS, Azure, Google, etcétera) y también por proveedores especializados, como CloudFlare, mediante una red de *proxies* inversos (denominados puntos de presencia o PoP) que tienen una caché de los contenidos, y sirven de puente entre los visitantes y la ubicación real de los datos estáticos a los que acceden.



Debido a la gran dificultad inicial que conlleva desplegar una aplicación en el *cloud*, se implementó el **servicio Beanstalk**, que es un PaaS en el que, partiendo de una aplicación compleja en una gran variedad de lenguajes (Java, .NET, PHP, Node.js o Python, o incluso contenerizada en un Docker), se genera una infraestructura de despliegue preparada para ese código en forma de un conjunto orquestado de máquinas virtuales (EC2), bases de datos (RDS), almacenamiento de versiones de código (S3), estructura de red (VPC), balanceadores de carga y condiciones de autoescalado.

Es un servicio sin coste; solo se paga por los recursos subyacentes que dan soporte a la infraestructura (máquinas virtuales EC2, almacenamiento S3, bases de datos RDS, etcétera) y que podríamos crear paso a paso nosotros mismos: simplemente, es una especie de asistente avanzado de despliegue, que realiza la tarea de creación y administración del entorno.

La infraestructura así desplegada podríamos crearla nosotros mismos paso a paso de igual forma, solo que nos ahorra ese trabajo, que puede resultar complejo si no se tienen los conocimientos para hacerlo. En definitiva, se trata de un servicio de orquestación de infraestructura, habitualmente utilizado para prototipado y despliegue rápido.

1.3. Servicios de computación sin servidor (serverless)

La evolución natural de los servicios *cloud* hacia entornos administrados y de despliegue/ desarrollo acelerado derivó en los **servicios de computación sin servidor (o serverless)** **Lambda y Fargate**.

Siempre que hablamos de las ventajas de la nube, se indica que es escalable y sin inversión inicial, y que solo se paga por lo que se usa. Eso es totalmente cierto, pero, cuando trabajamos con servidores en máquinas virtuales o contenedores, o a través PaaS, en mayor o menor medida tendremos el trabajo de planificar cuántos servidores necesitamos de base y definir las condiciones de escalado o suspensión cuando no se usan, así como seguridad y actualizaciones. Los escalados ascendentes y descendentes, además, llevarán tiempo (minutos), salvo que se tenga provisionada algo de infraestructura de antemano, asumiendo su coste

El siguiente paso, que profundiza en las ventajas de la nube, es la tendencia cada vez mayor a la **arquitectura sin servidor o serverless**, en la que el código de las aplicaciones se empaqueta en una API con tráfico regulado y se ejecuta igualmente en máquinas, pero que son provisionadas por el proveedor *cloud*, utilizadas y destruidas constantemente, de forma que se factura únicamente por los segundos de carga y ejecución.

El comienzo de esta tendencia parte de AWS, como pionero, con otra especialización del *cloud*, **las funciones como servicio o FaaS**, materializada desde 2014 en el servicio AWS Lambda.

Este tipo de tecnología es el caldo de cultivo perfecto para arquitecturas desacopladas, como los microservicios, y conlleva un desarrollo específico que afronta sus propios retos (como la gestión de las sesiones entre ejecuciones o la comunicación entre módulos), pero es el único camino posible si se desea rendimiento a toda prueba, a cualquier escala y a nivel global.

AWS Lambda se utiliza más para código estructurado en funciones sueltas que están almacenadas sin coste en un repositorio del proveedor (junto con librerías y otras dependencias que necesitemos), a la espera de un evento (una llamada explícita de otro proceso, una acción de usuario, una condición de servidor, etcétera) que dispare su ejecución. Sería la implementación de las FaaS, que soporta multitud de lenguajes y entornos de programación.

Lambda mantiene clústeres de millones de máquinas virtuales ya en funcionamiento, preparadas para la ejecución inmediata de las funciones, por lo que la primera vez que se invoca le asigna una y nos facturará por los segundos que esté en ejecución. Una vez terminada la ejecución, mantendrá la máquina un tiempo (sin facturarnos por ello), por si vienen más peticiones, y, si no es así, la destruirá.

Igualmente, en caso de llegar muchas peticiones concurrentes, asignará tantas nuevas máquinas en paralelo como sea necesario (a un ritmo máximo de 500 por minuto) y las irá destruyendo después, siguiendo la lógica anterior. Si tenemos necesidades especiales, como tiempos de respuesta inferiores a 100 ms o mayor capacidad de paralelización que la descrita, se pueden provisionar recursos temporal o permanentemente, con un coste adicional.

AWS Fargate es un servicio que da soporte virtualmente a contenedores (tanto de ECS como de EKS), de forma que no hay que mantener un clúster de máquinas EC2 subyacente, sino que se cobra por tiempo de ejecución del contenedor. Por tanto, no es un servicio independiente en sí mismo, sino una forma más evolucionada de ejecutar los servicios de contenedores bajo arquitectura *serverless*, usada normalmente para aplicaciones más estructuradas y complejas que en el caso de Lambda.

1.4. Servicios de computación en Azure

En Azure, el equivalente a AWS EC2 sería el servicio **Virtual Machines**, que, como su nombre indica, nos permite lanzar máquinas virtuales con un funcionamiento similar. En el caso de gestión de contenedores, tenemos los servicios **Azure Container Services** para la gestión de Docker simple, y **Azure Kubernetes Service** para estructuras más complejas orquestadas mediante Kubernetes.

Al igual que en AWS, estos servicios se pueden ejecutar sobre máquinas virtuales normales o mediante una arquitectura *serverless* similar a AWS Fargate. Dependiendo del escenario concreto, puede que un proveedor u otro sea más fácil de migrar o configurar, pero eso son detalles que escapan al ámbito de esta unidad.

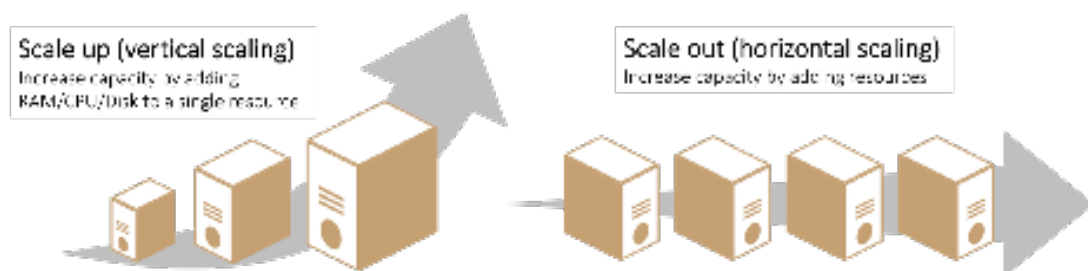
El equivalente en Azure a AWS Lambda para FaaS sería **Azure Functions**, con una estructura muy similar y que soporta igualmente gran cantidad de lenguajes y entornos de programación; **App Service** es su servicio base de alojamiento de funciones con escalabilidad limitada, y se puede pasar a *premium plan* si se requiere provisionar recursos y menor tiempo de respuesta.

La principal diferencia entre ambos es que la escalabilidad en App Service es bastante limitada, de forma que se agrupan las ejecuciones concurrentes en el mismo nodo (y no en nodos diferentes, como Lambda) desde su versión más básica, lo cual puede afectar a la ejecución, en caso de muchas llamadas simultáneas.

Respecto a los servicios de VPS y PaaS como AWS Lightsail y AWS Beanstalk, lo más parecido que podemos encontrar en Azure es el servicio **App Service**, que funciona principalmente mediante contenedores. Tiene un funcionamiento sencillo y bastante razonable, y las posibilidades de escalado son potentes. Por contra, se trata de un servicio mucho más empaquetado y limitado en opciones, con menor control sobre el despliegue y su infraestructura: es el peaje que se paga por su sencillez. Se tratará en un caso práctico.

02 Escalado vertical y horizontal en la nube

Un concepto fundamental de los sistemas en la nube es el «escalado horizontal» frente al «escalado vertical». Cuando hablamos de «escalar» un sistema nos referimos a aumentar su dimensión porque se ha quedado pequeño para nuestras necesidades. En el escalado vertical, simplemente aumentamos el hardware existente (mejores discos, memoria o procesadores), lo cual, en el fondo, es sencillo de implementar, pero puede ser caro e insuficiente a ciertos niveles. En el escalado horizontal se trata de añadir más recursos en paralelo, troceando y paralelizando las cargas de trabajo, lo que puede requerir adaptación en el desarrollo de las aplicaciones y no es tan sencillo, pero permite que la capacidad pueda crecer así ilimitadamente y añade al sistema redundancia y tolerancia a fallos.



Esquema 2. Diferentes filosofías de escalado, vertical y horizontal.

2.1. Escalado vertical de portales web convencionales en la nube

Supongamos el caso simple de un portal web en pila LAMP, como podría ser un WordPress, alojado en la nube mediante un servidor web en una máquina virtual, más un servidor de base de datos separado que le da soporte para repartir la carga. El sencillo esquema en AWS podría ser este:

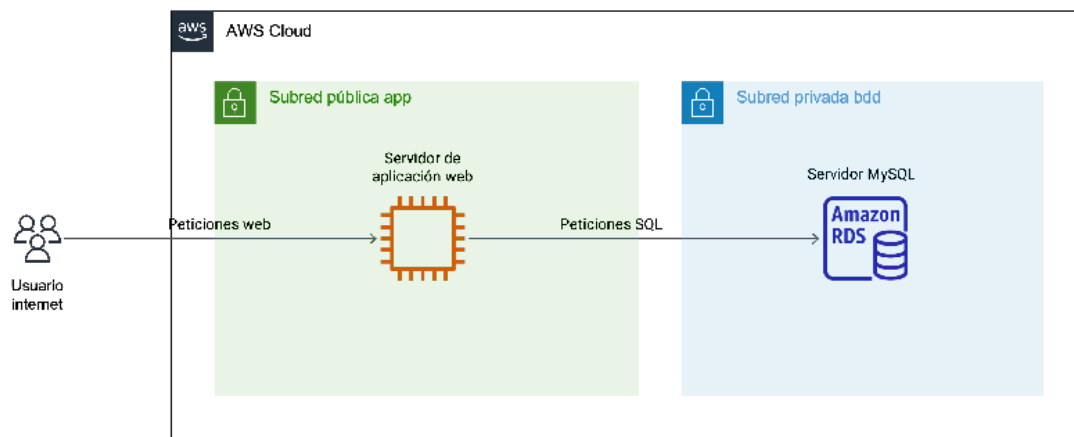


Figura 3. Despliegue típico de un portal web con base de datos, con mínimos requerimientos.

La máquina virtual EC2 en AWS se denomina «instancia», y se lanza con una determinada cantidad de memoria y potencia de CPU, que se factura por horas de uso. Lo mismo sucede con la base de datos, que normalmente se implementa en el servicio relacional o RDS, también en máquina virtual. Tienes los diferentes tipos de instancia y precios de uso por hora bajo demanda en los enlaces de ampliación.

En el ejemplo anterior, una opción habitual sería utilizar en ambos casos instancias **t3.micro**, que tienen dos vCPU (cores de un Intel Xeon Platinum) y 1 gigabyte de RAM. Para aplicaciones web, es recomendable utilizar instancias de la familia «t», que permiten «ráfagas» (familia *burstable* en inglés); de esta manera, acumulan potencia cuando no hay tráfico, que luego despliegan en los momentos pico.

¿Qué hacemos si los requerimientos de carga van aumentando? Una opción sencilla para adaptarnos puede ser el escalado vertical, es decir, mejorar la potencia de ambos servidores, en dos vertientes:

- » **Cambiar el tipo de instancia** a una mejor en términos de CPU y RAM. Por ejemplo, podemos pasar a una **t3.xlarge** de 8 vCPU y 16 gigabytes de RAM, multiplicando su coste, claro está. Para ello, habrá que reiniciar la instancia, con lo que el portal estará indisponible unos minutos.
- » **Cambiar la clase de disco de almacenamiento**. Esto, sobre todo, viene bien para las bases de datos, donde cambiar el disco de propósito general (gp2) por uno que tenga un número de operaciones de entrada-salida por segundo o IOPS alto y garantizado (io1/io2) aumentará notablemente su rendimiento, tanto en consultas como en operaciones de escritura. Para ello ni siquiera hay que reiniciar; son operaciones que se pueden hacer en caliente.

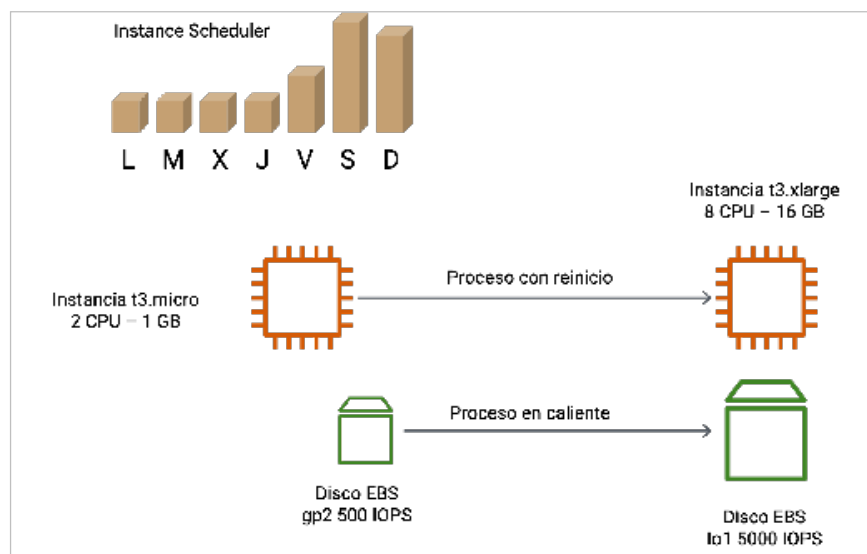


Figura 4. Escalado vertical de una instancia de servidor en tamaño (CPU y RAM) o en disco.

Una práctica muy extendida para controlar el coste, si el portal web tiene patrones de uso con grandes altibajos conocidos de antemano (por ejemplo, uso intensivo en horario de tarde/noche y fin de semana y muy bajo en horario de oficina), es realizar el cambio de tipo de instancia de forma programática conforme a esos patrones. Se puede hacer a través de *scripting* o llamadas a la API de AWS desde equipos locales o, mejor aún, utilizando un servicio alojado en la nube llamado AWS Instance Scheduler, que permite definir esas programaciones. De todo ello tienes detalle en los enlaces de ampliación.

Este tipo de escalado es trivial y suele ser el primer paso que se da si el aumento de requerimientos es razonable, pero si abusamos de ello llegará un momento en el que se disparará en coste y, lo que es peor, su utilidad puede ser limitada o nula para ajustarse a los requerimientos.

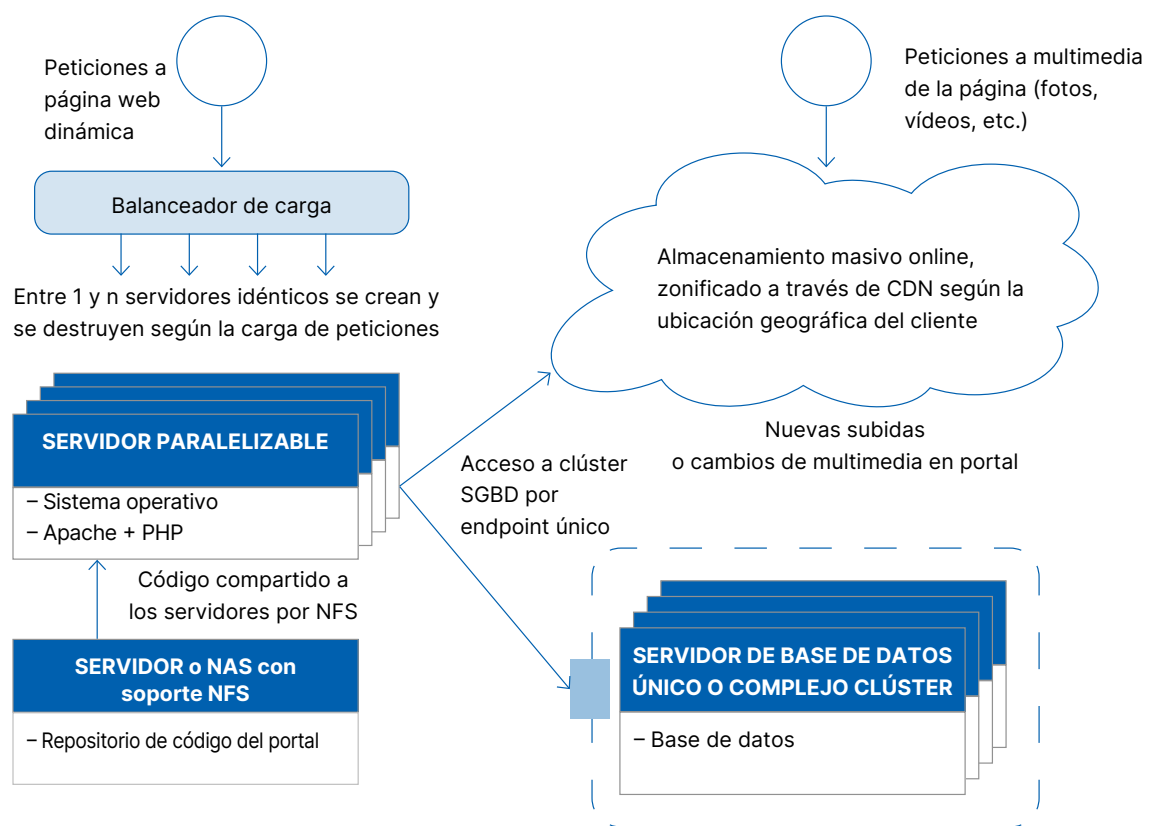
Cuando el sencillo escalado vertical no llega, es el momento de utilizar el escalado horizontal, mucho más complejo, pero que permite mantener una estructura que crece de forma virtualmente indefinida.

2.2. Escalado horizontal de portales web convencionales en la nube

Primeramente, hay que hacer notar que el escalado horizontal que vamos a tratar en este apartado es para estructuras clásicas de aplicaciones y portales típicos en pila LAMP llevadas a la máxima escalabilidad y alta disponibilidad en la nube.

En los siguientes apartados veremos que, en realidad, si queremos sacar el máximo partido del cloud y crecer de forma virtualmente infinita, deberemos plantearnos otro tipo de desarrollo más nativo para la nube, rompiendo las estructuras típicas de origen monolítico para pasar a usar tecnologías más preparadas para un escalado masivo, tales como *serverless* y microservicios.

Aun así, dentro de la nube podemos llegar bastante lejos escalando la estructura típica de un portal o aplicación web típicos en pila LAMP. Para ello, deberemos diseccionar y disociar la sencilla estructura anterior de servidor de aplicación más servidor de base de datos en varias partes. Tomando como referencia un portal web bajo pila LAMP como WordPress, este esquema muestra la estructura disociada en la nube:



Esquema 3. Estructura de un portal o aplicación web sobre pila LAMP desplegado de manera disociada en la nube.

- » Las peticiones PHP pueden ser atendidas indistintamente por un número indefinido de servidores idénticos que solo tienen el sistema operativo, servicios Apache/PHP y nada más, y se pueden escalar horizontalmente de forma sencilla (son instancias clónicas).
- » El código no está en los servidores del punto anterior, sino en un único repositorio centralizado que se comparte por NFS a estos; así, todos tienen el mismo código actualizado.
- » La multimedia de la web y los assets estáticos (como HTML, JavaScript, hojas de estilo, imágenes, vídeos, librerías de cliente, etcétera) se sirven directamente desde un almacenamiento masivo en la nube de bajo coste, sin servidor, con escalabilidad y alcance global mediante una red de distribución de contenidos o CDN.
- » La base de datos recibe las peticiones en un único punto de enlace o endpoint, que detrás encapsula una estructura escalable de uno o varios servidores, o un complejo clúster de ellos, según vayan creciendo los requerimientos, sin cambiar nada en el código.

Esta es una posible traducción de la figura anterior, para representar una aplicación web moderna que parte de pila LAMP mediante servicios de AWS, donde las peticiones web son manejadas por los servicios Route 53 para resolución DNS y CloudFront como CDN, **que las divide en estáticas y dinámicas según reglas definidas.**

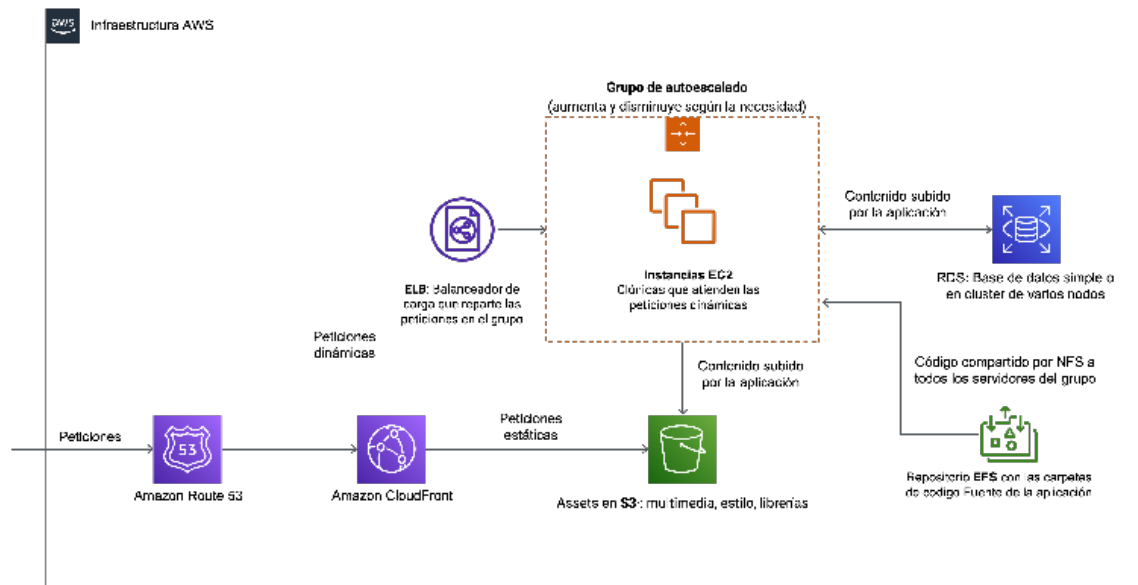


Figura 5. Implementación esquemática de una aplicación moderna bajo pila LAMP en AWS, partiendo de la figura anterior.

Vamos a detenernos un momento en las peticiones estáticas que son enviadas directamente al **almacenamiento masivo online S3**, del que hablaremos más adelante y en otras materias. En este servicio se pueden almacenar y recuperar objetos con una simple **API REST**, lo que permite activar un modo de uso para alojar el contenido estático y multimedia de los portales y aplicaciones modernas.

Las peticiones dinámicas son gestionadas por un balanceador de carga, una estructura de red automática que las reparte entre los servidores disponibles en un grupo de autoescalado. Este grupo aumenta y disminuye el número de servidores según las necesidades (tráfico, porcentaje de CPU, etcétera).

Para que todos los servidores del grupo ejecuten el mismo código, este se despliega en un almacenamiento EFS que es compartido a todos ellos (mediante NFS). La base de datos se implementa en el servicio relacional RDS (como caso más habitual), que puede incluir desde un único servidor hasta complejos clústeres de servidores con decenas de nodos, accedidos de igual manera sin cambiar nada en el código.

Todo esto resulta muy complejo de implementar, y más para un desarrollador de aplicaciones web que tiene limitados conocimientos sobre servidores y sistemas. Para intentar facilitarlo para nuestro perfil, es recomendable utilizar servicios de VPS como AWS Lightsail y servicios PaaS como AWS Beanstalk o Azure App Service, comentados en el apartado anterior y que puedes ver en los casos prácticos.

Estos servicios de orquestación y encapsulamiento abstraen hasta cierto punto la complejidad de los despliegues en la nube, si bien a partir de un cierto nivel de exigencia requerirá ajustes manuales complejos o migración a servicios más avanzados.

03 Servicios cloud para entornos de desarrollo

3.1. Introducción a los SaaS cloud

Los proveedores *cloud* tradicionalmente han funcionado como *IaaS* para migrar instalaciones locales por completo o en hibridación, adaptando sus desarrollos y su estructura a la nube. Conforme se va extendiendo su uso, las capacidades de computación escalable y en pago por uso del *cloud* abren nuevas posibilidades en campos como el *big data* o la inteligencia artificial y el *machine learning* en particular para cualquier empresa, pero la curva de aprendizaje y desarrollo para llegar a una situación concreta puede ser excesiva en muchos casos, y ahí es donde los SaaS especializados facilitan los procesos.

En esta última década, todos los proveedores han enfocado sus esfuerzos en desplegar un abanico de servicios SaaS creciente, con frecuencia varios de ellos cada año, a un ritmo frenético, a veces más generalistas o *core* y a veces más especializados, para facilitar procesos muy determinados.

Por ejemplo, supongamos el caso de una empresa que quiere procesar a nivel global enormes cantidades de vídeo y audio de cámaras de seguridad, almacenarlos y ponerlos a disposición de los clientes. Una opción sería usar un clúster de servidores en la nube, desarrollar un software que recoja las señales, las procese y las almacene de forma coordinada en una compleja infraestructura global en el proveedor que prefiera. Pero puede ser complejo de implementar, y el cálculo de costes para determinar el margen de beneficio, aunque sea un pago por uso flexible, puede ser complicado.

Sin embargo, en los proveedores *cloud* actuales existen SaaS especializados para ello: uno para absorber y retener flujos multimedia de dispositivos, procesos *serverless* para procesar esos flujos, y servicios de almacenamiento masivo al menor coste posible en función de unos plazos y reglas de acceso, con un cálculo de costes trivial, que permite establecer un margen de beneficio con el que se sabe cuánto cobrar al cliente y ser competitivo.

Tanto en este caso en concreto como en el desarrollo de un gran aplicativo, puede abstraerse una buena parte de la complejidad de este mediante servicios empaquetados accesibles y fácilmente integrables a través de una API a la que dirigir las peticiones y obtener los resultados.

Incluso donde la funcionalidad aportada por el SaaS no sea un elemento central, su uso permite centrarse en añadir características innovadoras; en mejorar el producto para diferenciarse de la competencia. Así, una vez que la empresa del ejemplo tiene controlada toda la gestión de los elementos multimedia para cualquier número de clientes globales de forma escalable, puede utilizar más servicios SaaS de IA para reconocer qué alertas son erróneas, reconocer rostros en las grabaciones o desechar situaciones en las que no haya riesgos. Todo ello no es parte central del aplicativo inicial, ni es imprescindible, pero le da un valor añadido, con un coste predecible y escalabilidad total según el éxito que tenga el producto, con riesgos muy limitados.

3.2. Principales SaaS de interés en los proveedores cloud

Los SaaS en los proveedores *cloud* son un complejo universo creciente, a una media de ocho servicios nuevos —o especializaciones de los existentes— cada año. Es imposible conocerlos todos en profundidad, pero sí que es interesante saber cómo están clasificados para decidir cuáles pueden ser de nuestro interés para una situación y unas necesidades concretas.

» **Servicios generalistas o «core».** Son los servicios principales, los cimientos del ecosistema *cloud* que sirven de base para el resto, identificados en negrita en la tabla. Por similitud semántica, se han añadido a la tabla algunas especializaciones posteriores de gran interés que se han convertido en fundamentales.

SERVICIO	AWS	AZURE
Computación, máquinas virtuales, <i>serverless</i>	EC2 , Batch ECS, ECR, EKS Lambda , Beanstalk , Outposts, Step Functions	Virtual Machines , Batch Container Registry, AKS Functions , App Service , Stack, Logic Apps
Almacenamiento de bloques y de objetos	EBS , S3 , EFS	Disk Storage , Blob , Files
Base de datos SQL y NoSQL	RDS-Aurora , DynamoDB , Neptune, ElastiCache Redshift , Lake Formation, Athena	SQL Database , CosmosDB , Cache for Redis Synapse , Data Share, Data Lake Analytics

Redes, entrega de contenido y escalabilidad	VPC , VPN Gateway, Route 53 , Direct Connect PrivateLink, VPC Endpoints CloudFront Load Balancer , Autoscaling , OpsWorks	Virtual Networks , VPN Gateway, DNS , ExpressRoute, PrivateLink, Private Endpoints Azure CDN Load Balancer , VM Sets , Automation
Monitorización, administración y seguridad	Cloudwatch , Cloudformation Cloud Trail, Cognito , IAM KMS Web Application Firewall, Shield	Monitor , Application Insights , Resource Manager Activity Log, Azure AD Key Vault Web Application Firewall, DDos Protection

Tabla 1. Servicios centrales de AWS y Azure (en negrita) y algunas especializaciones.

» **Servicios especializados.** Se centran en aspectos muy concretos y variopintos, y ofrecen servicios empaquetados y accesibles normalmente mediante API REST, donde en muchas ocasiones se pueden construir procesos donde la entrada de un servicio es la salida coordinada de otro, encadenándose de forma compleja decenas de veces.

SERVICIO	AWS	AZURE
IA – <i>machine learning</i>	SageMaker, Lex, Polly Rekognition, Alexa Skills	Machine Learning, Speech Cognitive, Bot Framework
Análisis de datos y visualización	Kinesis, Quicksight, Elasticsearch	Data Lake Analytics, PowerBI Elastic Azure
Migración de datos	Database Migration Service Import/Export Disk, Snowball	Database Migration Service Import/Export, Data Box
Colaboración entre equipos de desarrollo e integración continua	CodeDeploy CodeCommit CodePipeline CodeBuild	DevOps
Automatización y orquestación de infraestructura de despliegue	CLI CloudFormation OpsWorks CloudShell	CLI Automation Resource Manager

Desarrollo de microservicios	App Mesh, Developer Tools, SQS SNS, Kinesis	Service Fabric Mesh, Developer Tools, Queue Storage Service Bus, EventHub
IoT y procesamiento de flujos de datos	Timestream, IoT core, Greengrass Kinesis Firehose y Streams	Time Series, IoT Hub IoT Edge, Event Hubs
Procesamiento big data	EMR, Data Pipeline, Glue	Data Explorer, HDInsight
Miscelánea: computación cuántica, gaming	Amazon Braket GameLift	Azure Quantum PlayFab

Tabla 2. Servicios especializados de AWS y Azure.

¿Cómo se llega este nivel de especialización? Los proveedores *cloud* mantienen repositorios de código (por ejemplo, en Github) y siguen con interés los desarrollos y usos de diferentes clientes y sus necesidades: en cuanto hay algo de interés a lo que se le ve potencial, rápidamente se implementa un detallado SaaS al respecto (a veces partiendo del mismo repositorio), con un proceso continuo de mejora y ampliación.

3.3. Servicio de almacenamiento masivo S3

Si hay un SaaS que podamos considerar el centro de gran parte de la infraestructura y despliegue de desarrollo, ese es S3. Sabemos que ofrece espacio de almacenamiento masivo para cualquier tipo de datos, una alternativa a construir, mantener y utilizar tus propios sistemas de almacenamiento, sin límites definidos (es paradigmático el uso de él que hace Apple, que tiene alojado allí todo el contenido de iCloud a nivel mundial: le sale más a cuenta que mantener sus propios centros de datos).

Se basa en **contenedores llamados «cubos» o buckets**, cuyo nombre debe ser único, donde se almacenan **objetos** que se componen de los datos en sí y cualquier cantidad de metadatos asociados. Los objetos se pueden organizar en carpetas por diversos motivos prácticos, pero no es esencial. Cada objeto se almacena o recupera con una simple, y cualquier pequeño cambio en un objeto supone actualizarlo al completo.

Lo importante, de cara al desarrollo de aplicaciones, es que, en realidad, el servicio S3 es muy complejo y tiene múltiples ramificaciones con casi todos los demás servicios de AWS de una u otra forma (decenas de ellos), por lo que, en uno u otro momento, habrá que pasar por él.

Sobre objetos y *buckets* se pueden definir eventos, de forma que cualquier acción sobre ellos (como añadir, actualizar, eliminar, etcétera) puede **desencadenar eventos conectados con el resto de servicios cloud**.

Por ejemplo, pensemos en una red social centrada en imágenes y vídeos. Cada vez que un usuario sube una imagen o vídeo, se almacena en un *bucket* determinado. Puede programarse un evento que dispare un código *serverless* (servicio Lambda) que genere *thumbnails* y previsualizaciones de vídeo en distintos tamaños para todo archivo que se suba al *bucket*, enviándolo a otro *bucket* diferente.

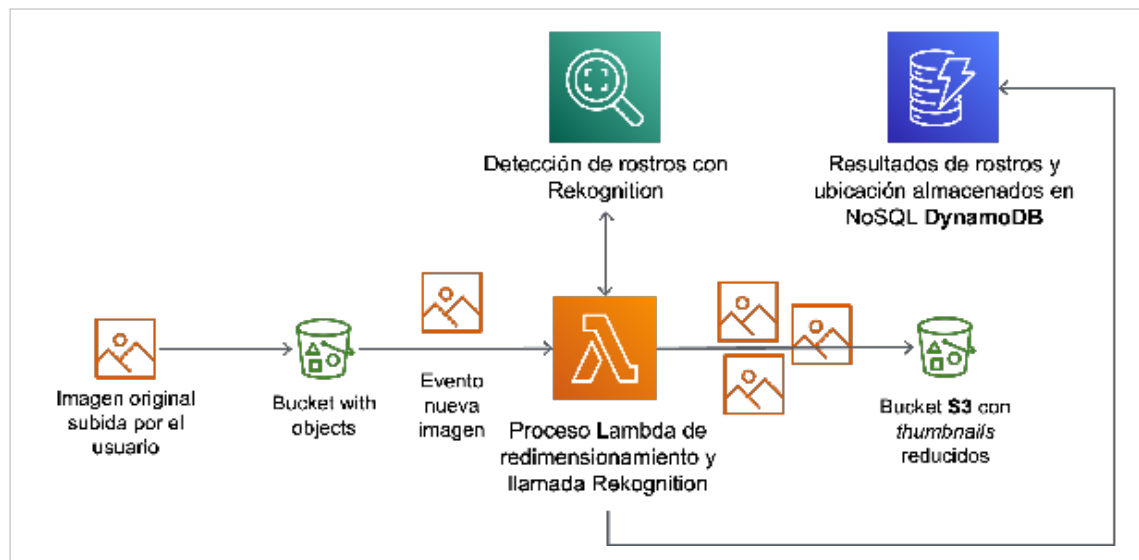


Figura 6. Buckets de S3 que disparan eventos de procesamiento Lambda sobre imágenes que se almacenan en ellos.

Igualmente, el mismo código *serverless* puede enviarlo al servicio Rekognition para que reconozca los rostros de las personas que aparecen y los almacene en una base de datos para su etiquetado en fotos o vídeos posteriores. Todo esto no son ejemplos hipotéticos, **son formas de trabajo que se están usando desde hace años en aplicaciones modernas implementadas de forma nativa en el cloud**, y que tienen a S3 y su interconexión con el resto de servicios como eje principal.

Estos y otros conceptos relacionados se tratarán en casos prácticos en esta unidad.

3.4. Estándares de facto y decisión sobre proveedores

Como se ha visto, el abanico de servicios *cloud* es apabullante, con decenas de servicios principales y cientos de servicios especializados (las tablas anteriores no son completas; hay muchos más).

No obstante, se están estandarizando unas categorías de servicios *cloud* en todos los ámbitos, liderados por AWS como pionero y principal proveedor, aunque a veces lo es Azure y en el futuro puede que lo sean otros.

Esta circunstancia hace que cualquier servicio *cloud* imaginable esté implementado por los principales proveedores, como bien se puede ver en las tablas anteriores, con correspondencias y adaptaciones, e igual se podría hacer para Google u otros, en mayor o menor medida.

A una velocidad de vértigo, partiendo de las necesidades que van surgiendo de forma orgánica entre los clientes y desarrolladores, se están creando **estándares y categorías cloud «de facto»**.

Esta circunstancia lleva a la conclusión de que, en la práctica, **importa poco el proveedor concreto de cloud computing que se estudie**; todos los proveedores presentes y futuros implementan servicios equivalentes en categorías equivalentes que cubren necesidades de una forma más o menos idónea según las condiciones preexistentes y los casos concretos. También es un hecho habitual aprender a usar la nube con un proveedor concreto, que constituye **una «lengua materna» en la que se desenvolverse con soltura**. Posteriormente, se pueden dominar otros proveedores, pero siempre acabaremos «traduciendo» los procesos a nuestra referencia mental, como pasa con una lengua adquirida: nunca nos parecerá natural ni razonaremos en ella.

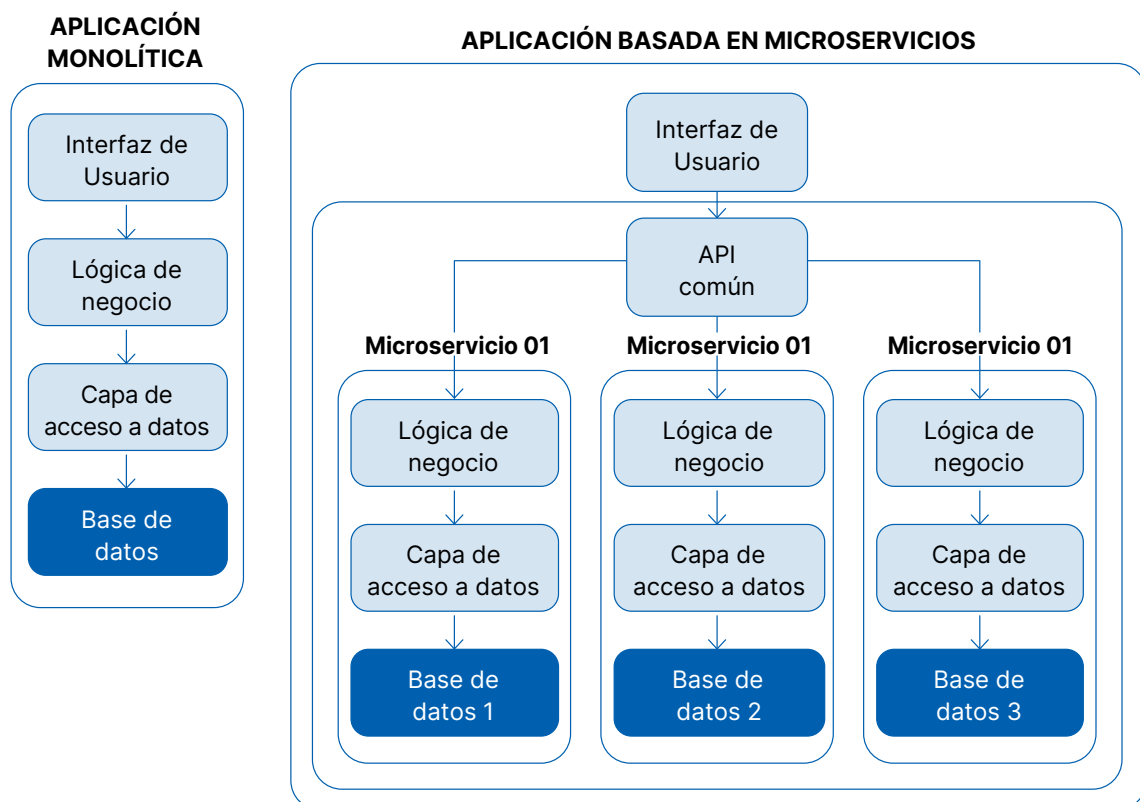
En el ámbito empresarial concreto, se usará un proveedor u otro, de acuerdo con aspectos de coste económico, fiabilidad, integración con lo preexistente (aplicaciones, licencias, etcétera), cobertura de necesidades actuales, proyección futura y sinergias que proporcione con otros aspectos —no técnicos— de la empresa: no hay un proveedor universal que a todo el mundo le venga bien.

04 Microservicios

4.1. Arquitectura

Las aplicaciones tradicionales integran todas sus capas en una estructura **monolítica** en la que todas las funcionalidades están fuertemente interconectadas. Cuando las aplicaciones crecen en complejidad, resulta difícil realizar modificaciones o añadir nuevas funcionalidades por parte de los equipos de desarrollo: por ejemplo, un fallo crítico en uno de los módulos puede provocar la caída completa de la aplicación.

La arquitectura basada en microservicios consiste en dividir una aplicación en un conjunto de servicios **independientes** que se comunican a través de **API** bien definidas. Cada uno de dichos microservicios es mantenido por un equipo independiente. Dado que un microservicio se ocupa de una parte específica de la aplicación, su desarrollo se ve simplificado, y por lo tanto disminuye el tiempo necesario para su puesta en producción. Además, al estar su funcionalidad separada, un fallo en alguno de los servicios no provoca la caída de toda la aplicación.



Esquema 4.

Algunas de las **ventajas** que ofrece este modelo de desarrollo son:

- | | |
|--------------------------------------|---|
| Agilidad | Los microservicios ofrecen funcionalidades más sencillas de implementar, por lo que su desarrollo es más sencillo y necesita menos tiempo. |
| Escalado | Cada microservicio puede ser escalado de manera independiente, en función de la carga de trabajo que soporte en cada momento. |
| Libre elección de tecnologías | Dado que los microservicios operan de manera independiente, es posible utilizar diferentes tecnologías para cada uno. Ello ofrece una mayor flexibilidad a los equipos desarrolladores para elegir las tecnologías más apropiadas o con las que se sientan más cómodos. |

Reutilización de código

La posibilidad de dividir la aplicación en componentes más pequeños y especializados permite el desarrollo de **bibliotecas de componentes** que pueden ser utilizados en otras aplicaciones diferentes.

Resiliencia y tolerancia a errores

Un fallo en un microservicio normalmente no afecta a toda la aplicación, sino a alguna de sus características: de esta manera, la aplicación, aunque con funcionalidad degradada, puede seguir operando.

4.2. Implementación

Una de las ventajas de utilizar una arquitectura basada en microservicios es que cada uno de ellos puede utilizar una **tecnología de backend** diferente. Por ejemplo, un microservicio podría utilizar servidores convencionales utilizando **instancias de EC2**, mientras que otro podría utilizar **contenedores**, y otro, tecnologías **serverless**, como **AWS Lambda**.

Del mismo modo, cada microservicio puede utilizar un sistema de **almacenamiento de datos** diferente. Por ejemplo, un microservicio podría utilizar una **base de datos relacional**, mientras que otro podría optar por una base de datos **NoSQL**, como **AWS DynamoDB**. Es habitual que cada microservicio disponga de su propia capa de datos, aunque también es posible optar por un sistema de almacenamiento centralizado.

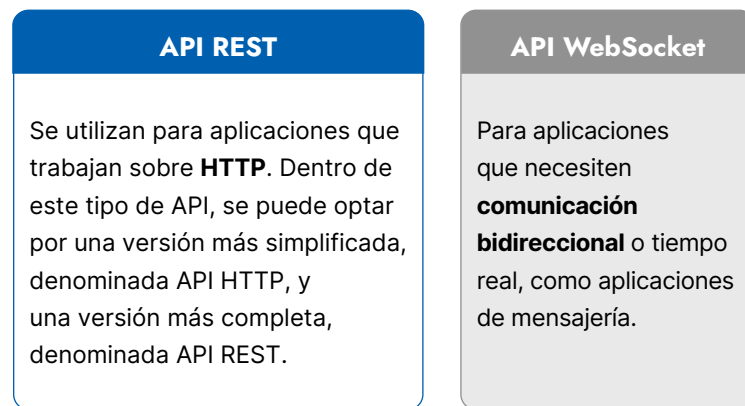
Como cada microservicio está implementado de manera independiente, es necesario utilizar algún mecanismo que permita la **comunicación** entre ellos cuando deban interactuar entre sí. Además, es necesario proporcionar un **punto de acceso común** para que los clientes, las aplicaciones de **frontend**, puedan interactuar con la aplicación de manera **transparente** sin tener que preocuparse de averiguar con qué microservicio deben interactuar. Dicha comunicación se suele realizar a través de una **API**, que proporciona una **interfaz común** de acceso y comunicación.

Por otro lado, dado que cada microservicio se suele encargar de realizar únicamente las operaciones de almacenamiento que le competen, resulta también necesario implementar algún tipo de mecanismo que garantice la correcta realización de **transacciones** y garantice la **consistencia** de los datos almacenados. Para ello, se suele utilizar un servicio de **orquestación** que se encargue de coordinar las operaciones de las diferentes transacciones y de **revertir** los cambios en el caso de que se produzca algún error durante el procesamiento.

4.3. Acceso y comunicaciones

Una aplicación basada en microservicios necesita de un punto de entrada común que ofrezca a los clientes una **interfaz unificada** con la que actuar y que además permita a los diferentes microservicios **interactuar entre sí**. Dicha interfaz se debe encargar de recibir las peticiones y **redirigirlas** a los microservicios correspondientes para su procesamiento. AWS proporciona el servicio **API Gateway** para esta tarea. Amazon API Gateway es un

servicio que permite **crear y gestionar APIs**. Se trata de un servicio **gestionado**, por lo que puede **escalar automáticamente** en función del número de peticiones que soporte en cada momento. Este servicio proporciona **dos tipos de API**:



Esquema 5.

Además de actuar como punto de entrada común a la aplicación, API Gateway ofrece también las siguientes **características**:

Escalado automático	Capaz de trabajar con un gran número de peticiones simultáneas sin necesidad de realizar ninguna configuración adicional.
Gestión de tráfico	Posibilidad de priorizar y redirigir el tráfico.
Soporte CORS	Posibilidad de configurar el servicio para permitir el acceso desde dominios diferentes al de la aplicación.
Autorización y control de acceso	Compatible con varias técnicas de control de acceso para las distintas rutas de la API.
Versionado	Permite crear diferentes versiones de la API y gestionarlas de manera independiente.
Soporte de diferentes backends	Es posible utilizar diferentes tecnologías de <i>backend</i> , como AWS Lambda, contenedores o instancias de EC2.

API Gateway es un servicio de **pago por uso**: se tarifica por **número de peticiones realizadas** y por cantidad de **datos transferidos**.

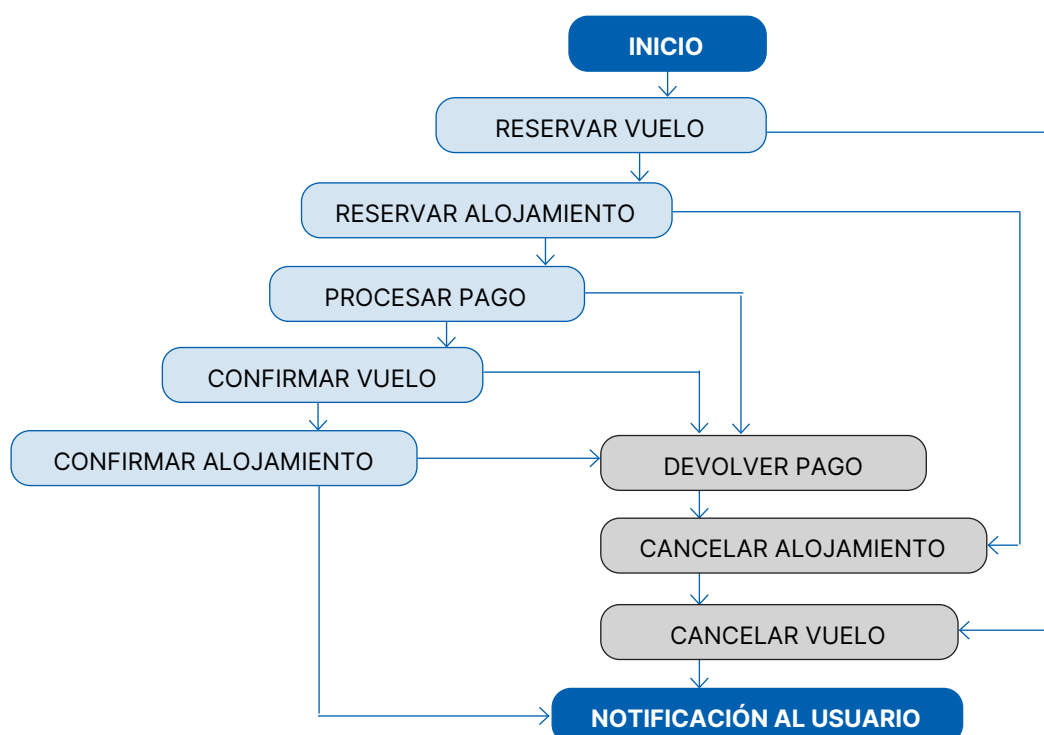
El proveedor **Azure** ofrece un servicio similar a AWS API Gateway, denominado **Azure API Management**.

4.4. Orquestación

Tal como se ha comentado anteriormente, la arquitectura basada en microservicios puede presentar problemas a la hora de ejecutar **transacciones** complejas que impliquen varios pasos que dependan unos de otros. Consideremos un ejemplo de aplicación de reservas vacacionales. Dicha aplicación se ha implementado utilizando microservicios, cada uno de los cuales se encarga de realizar una tarea independiente, como, por ejemplo:

- » Reserva de vuelo.
- » Reserva de alojamiento.
- » Procesamiento del pago.
- » Confirmación y reserva del vuelo con la aerolínea.
- » Confirmación y reserva del alojamiento con la empresa hotelera.
- » Notificación al usuario.
- » Devolución del pago en caso de error.
- » Cancelación de la reserva de vuelo a petición del usuario o en caso de error en la confirmación de la aerolínea.
- » Cancelación de la reserva de alojamiento a petición del usuario o en caso de error en la confirmación de la empresa hotelera.

A continuación, se muestra un ejemplo del diagrama de estados de una posible implementación de la solución:



Esquema 6.

Una vez que el usuario ha seleccionado las opciones correspondientes en la interfaz de usuario y solicita la gestión de la reserva, la aplicación debe realizar las acciones indicadas, en el orden seleccionado. Por ello, es necesario crear código de sincronización para realizar las tareas de **orquestación**. Para esta tarea, AWS proporciona el servicio **Step Functions**. AWS Step Functions permite construir aplicaciones a través de componentes individuales en forma de **máquina de estados**. Para ello, proporciona una **interfaz gráfica** y un **lenguaje estándar** que permite definir acciones como ejecuciones en **serie** o **paralelo**, **gestión de errores** o **ramas condicionales**. El uso de este servicio disminuye el acoplamiento fuerte (*tight coupling*) entre los componentes del sistema, y favorece su reemplazo rápido en caso de actualización o reparación.

AWS Step Functions soporta diferentes tecnologías de **backend**, como AWS Lambda, instancias de EC2 o contenedores.

El proveedor **Azure** ofrece un servicio similar a AWS Step Functions, denominado **Azure Logic Apps**.

4.5. Control de acceso

Las aplicaciones monolíticas tradicionales utilizan **sesiones**. Mediante este mecanismo se **evita** tener que enviar las credenciales de acceso (normalmente usuario y contraseña) con **cada petición**. En su lugar, se crea un **identificador de sesión** que se asocia con el usuario autenticado: siempre que las futuras peticiones incluyan una *cookie* con dicho identificador de sesión, se considerarán peticiones autenticadas.

Sin embargo, las aplicaciones basadas en microservicios se encuentran **distribuidas** entre diferentes servidores, por lo que, si se desean utilizar sesiones, es necesario utilizar algún sistema de **almacenamiento compartido**, como una base de datos de tipo caché, para almacenar las variables de sesión que identifiquen las peticiones realizadas por un mismo usuario.

Como alternativa, es posible utilizar mecanismos de autenticación basados en **tokens** y tecnologías basadas en **OAuth** y **OpenId**. De esta manera, se delega la tarea de autenticación al **frontend**, que se encarga de realizar la petición de autenticación a un servicio especializado, obtener un *token* de acceso y utilizar dicho *token* para autenticar las peticiones a la aplicación.

AWS proporciona el servicio **AWS Cognito** para esta tarea. Se trata de un servicio **gestionado** que se integra con **API Gateway** para automatizar las tareas de gestión y autenticación de usuarios. Tal como ocurre con el resto de servicios gestionados estudiados, se trata de un servicio de **pago por uso** (en este caso, pago por usuario activo) que es capaz de **escalar** automáticamente en función del número de peticiones simultáneas.

A continuación, se muestra un ejemplo de funcionamiento de un proceso de autenticación de una aplicación basada en microservicios:

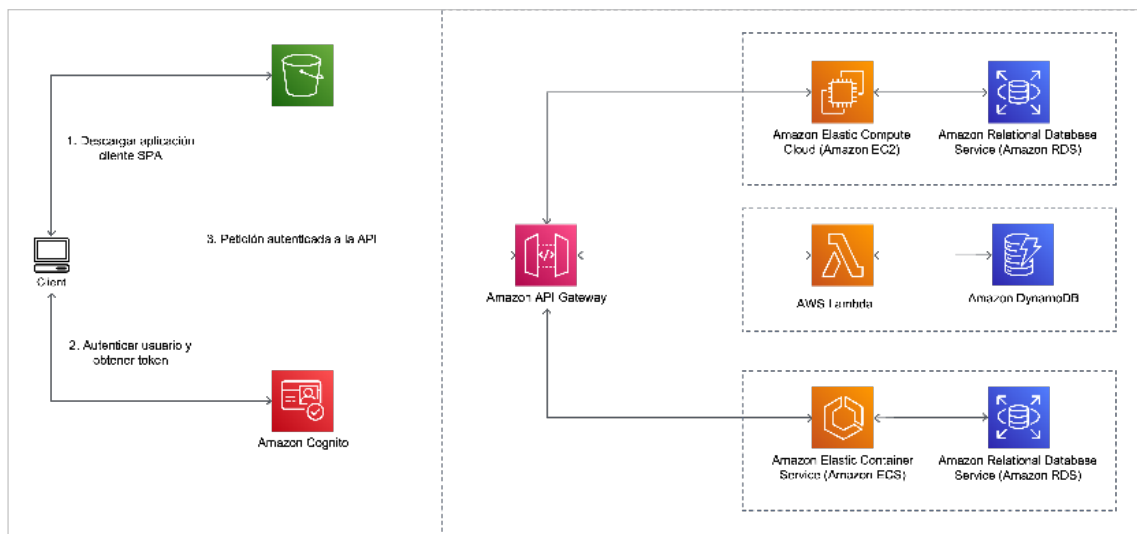


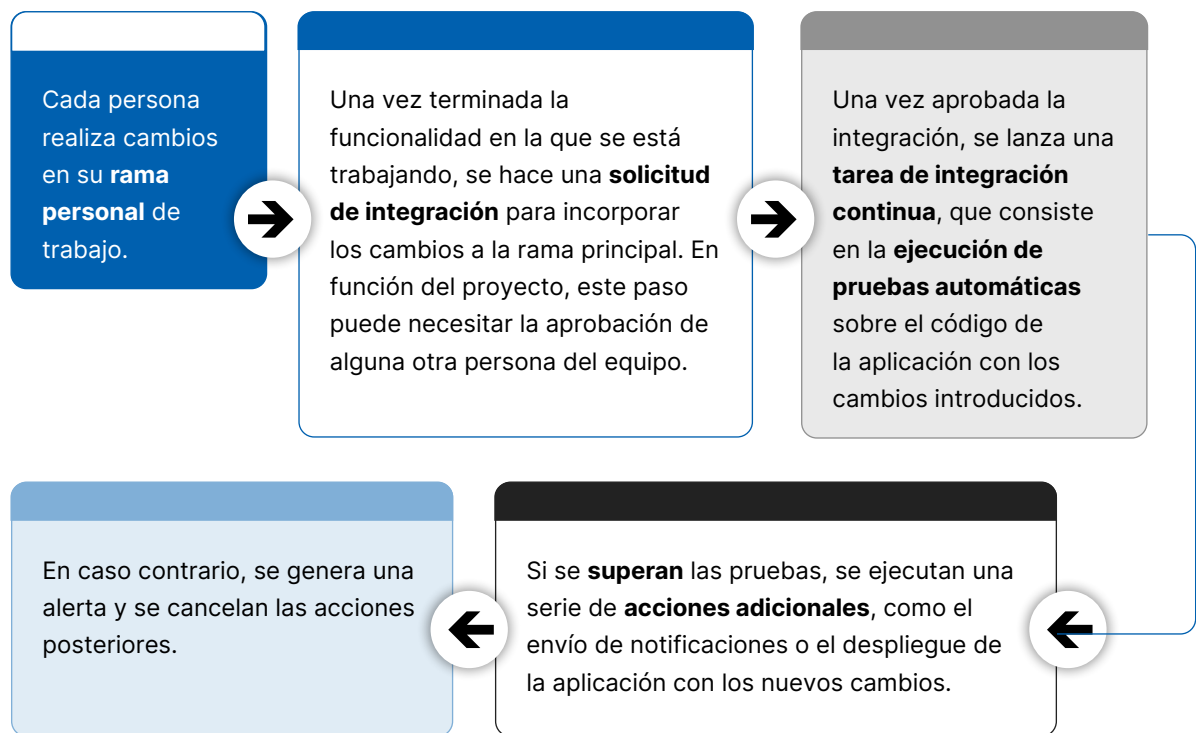
Figura 7.

El proveedor **Azure** ofrece su propio servicio de autenticación y gestión de usuarios, denominado **Azure Active Directory**.

05 Código en la nube

5.1. Integración continua

La **integración continua**, o *continuous integration* en inglés, es una metodología de trabajo que consiste en **incorporar los cambios de código** producidos por las personas desarrolladoras de un proyecto en una **rama de código común** de manera **frecuente**. El procedimiento más habitual consiste utilizar un sistema de **control de versiones**, que suele ser **git**, con un **repositorio común**. Dicho repositorio está formado por una **rama principal**, que almacena el código listo para ser llevado a **producción**, y un conjunto de **ramas de desarrollo** utilizadas por los participantes del proyecto para realizar sus cambios. El **flujo de trabajo** que se suele utilizar es el siguiente:



Esquema 7. Flujo de trabajo.

Esta metodología suele combinarse con la metodología de **desarrollo guiado por pruebas**, o *test-driven development* (**TDD** en inglés). En ella, cada persona desarrolladora define **nuevos test** para comprobar la funcionalidad que está desarrollando. La tarea de integración continua se encarga de ejecutar **todos los test**, antiguos y nuevos, para garantizar que los nuevos cambios no afectan a las funcionalidades anteriormente desarrolladas.

En ocasiones, es interesante realizar la **publicación** o **paso a producción** de los nuevos cambios de **manera automática** una vez que se haya comprobado que superan las pruebas. En este caso, el concepto de integración continua se **amplía** y recibe el nombre de **despliegue continuo** o **entrega continua**. Esta metodología **añade los siguientes pasos**, que se ejecutarán únicamente **si se supera** la fase de ejecución de **pruebas**:

- » **Compilación** o generación de la aplicación.

- » **Despliegue**. Una vez compilada la aplicación, es posible automatizar su **despliegue** o publicación en un servidor. Si el proceso de despliegue es manual, se habla de *continuous delivery*, o entrega continua; si el proceso de despliegue es automático, se habla de *continuous deployment* o despliegue continuo.

5.2. Herramientas de integración y despliegue continuos

AWS proporciona un conjunto de servicios para las tareas de integración y despliegue continuos. Dichos servicios son:

AWS CODECOMMIT

Servicio de **control de versiones** basado en repositorios Git. Este servicio es gratuito para los primeros cinco usuarios.

AWS CODEBUILD

Servicio de **integración continua** que permite ejecutar test, compilar código fuente y producir archivos ejecutables. Los resultados de ejecución pueden almacenarse en AWS S3. El coste del servicio se calcula por la cantidad de **tiempo de procesamiento** medido en **minutos** y por el **tipo de instancia EC2** que se elija para realizar la tarea.

AWS CODEDEPLOY

Servicio encargado de realizar el **despliegue** de la aplicación en servicios de AWS, como EC2, Lambda o Fargate. El coste del servicio es **gratuito** si el despliegue se realiza en servicios de AWS, como EC2, Lambda o ECS.

AWS CODEPIPELINE

Servicio que permite la creación de **flujos de trabajo** que conecten los servicios anteriores. Mediante CodePipeline, es posible crear un flujo que, cada vez que detecte un nuevo **commit** en CodeCommit, lance un proceso en CodeBuild y despliegue el ejecutable mediante CodeDeploy. El coste del servicio se calcula por **número de flujos de trabajo activos**, y son gratuitos los primeros 30 días de cada flujo.

Esquema 8.

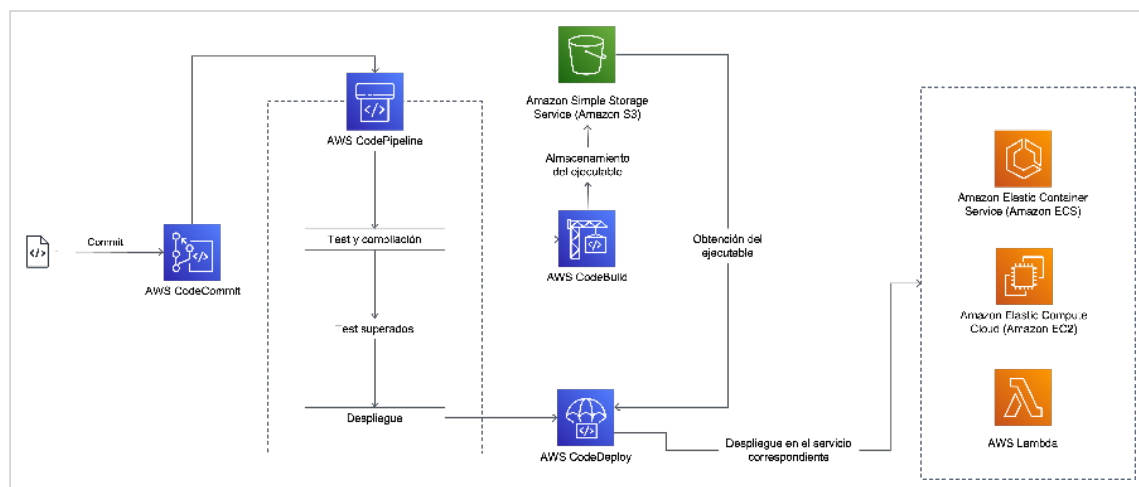


Figura 8.

El proveedor **Azure** ofrece un conjunto de servicios similar dentro de su paquete **Azure DevOps**. En él se pueden encontrar servicios como **Azure Repos**, semejante a AWS CodeCommit; **Azure Pipelines**, parecido a AWS CodePipeline y AWS CodeDeploy, y **Azure TestPlans**, similar a AWS CodeBuild. Además, proporciona el servicio **Azure Artifacts** como **repositorio de paquetes** compatible con los lenguajes de programación más populares (Java, .NET, NodeJS y Python).

Por último, **GitHub** incorpora también su propio servicio de integración continua, denominado **GitHub Actions**, que se integra de manera muy sencilla con los repositorios de código de dicha plataforma.

5.3. Editores en la nube

El trabajo con lenguajes de programación exige la utilización de un **entorno de desarrollo** formado por un **editor** de código, un **entorno de ejecución**, un conjunto de **librerías** y una serie de **herramientas adicionales**, como **control de versiones**, **terminal de comandos** o herramientas de **test**.

La **instalación** y **configuración** de un entorno de desarrollo suele ser un proceso **costoso**, que además se debe realizar en **cada equipo** que se vaya a utilizar como equipo de desarrollo. Por otra parte, dicho entorno puede ser utilizado únicamente por **una persona** a la vez.

Como alternativa, AWS ofrece el servicio **AWS Cloud9**, un entorno de desarrollo **basado en la nube** que permite el acceso a todas sus funcionalidades a través de un **navegador web**. De esta manera, se puede hacer uso del entorno de desarrollo desde **cualquier dispositivo** que disponga de un navegador. Además, ofrece funcionalidades de **trabajo en equipo**, como el **chat** o la **edición colaborativa** en tiempo real.

Cloud9 ofrece compatibilidad con los lenguajes JavaScript, Python, PHP, Ruby, Go y C++. Ofrece además integración nativa con los servicios de integración continua de AWS.

El servicio Cloud9 se ejecuta sobre una **instancia de EC2**. Puede utilizarse con una instancia **dedicada**, es decir, creada expresamente para el editor, o una instancia **existente** accesible vía SSH. El único **coste** asociado al servicio corresponde al de la **instancia EC2** sobre la que se ejecute.

5.4. Desarrollo de aplicaciones en la nube

El servicio **AWS Amplify** permite crear **aplicaciones web completas** con lógica de cliente y servidor. Utiliza una **interfaz visual** para crear el **interfaz de cliente** y el **modelo de datos**. Además, ofrece, entre otras, las siguientes **funcionalidades** adicionales, que pueden añadirse de manera opcional a la aplicación:

- » Autenticación.
- » Almacenamiento de objetos.
- » Análítica de datos.
- » Notificaciones.

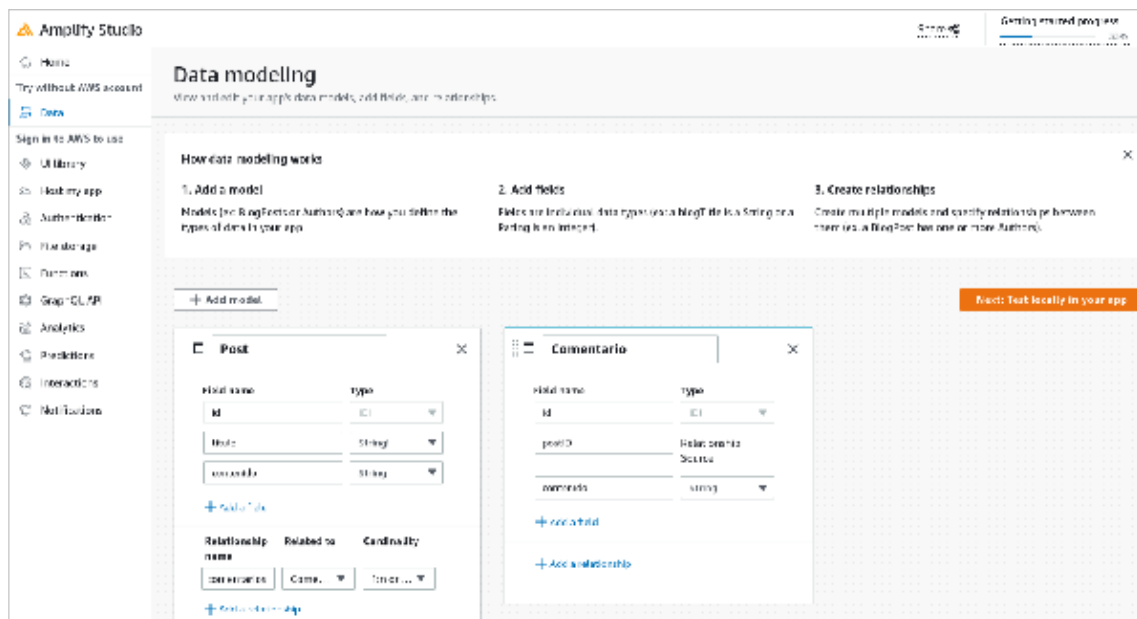


Figura 9.

AWS Amplify actúa como un **asistente** que se encarga de crear y conectar los servicios AWS necesarios para implementar las funcionalidades seleccionadas. Por tanto, no tiene coste adicional: solo se **tarifica** por los **servicios AWS utilizados de manera subyacente**. El código de la aplicación se almacena en un **repositorio** que puede estar alojado en cualquier proveedor, como GitHub, GitLab o AWS CodeCommit. AWS Amplify se encarga de las tareas de **integración y despliegue continuos**, por lo que cada actualización en la rama elegida para el despliegue desencadenará un flujo de trabajo que culminará con la publicación de la aplicación actualizada.



VÍDEOLAB

Para consultar el siguiente vídeo escanea el código QR o [pulsa aquí](#).

UAX FP